Why Can Large Language Models Speak and Think Like Humans?

Hang Li, Shaohua Zhang, Yuan Lin

1 Introduction

We use Large Language Models (LLMs) every day. A clear impression is that they truly seem to understand our language, although hallucinations do occur from time to time. On the other hand, observing the chain-of-thought output by LLMs—that is, the linguistic representation of their reasoning process—we feel that they can truly think like humans.

This article discusses: **What kind of capabilities are the language and thinking abilities of LLMs? How are these capabilities formed through their implementation principles, methods, and even mechanics?** (See Figure 1)

LLM technology is created by humans, and its implementation principles are clear, but its mechanics (internal working mechanisms) is still not fully understood. LLMs are extremely large in scale, and their mechanics are extremely complex, posing significant challenges to research on their capabilities.

Since the advent of ChatGPT, there has been extensive research on the mechanisms and properties of LLMs, particularly in recent years on mechanics (or interpretability). These works have provided answers to a certain degree on this core AI topic from different perspectives. However, many questions remain for future research.

This article summarizes the basic principles and implementation methods of LLMs, and provides a brief introduction to research on LLM mechanics, including ByteDance's work on LLM memory mechanisms. Based on this, we present our own views on how LLM capabilities are formed. This article contains three main arguments.

- **What LLMs learn are patterns of language and reasoning, and importantly, they have learned higher-order patterns.** LLM learning belongs to machine learning, and what is learned is essentially statistical regularities in data, or patterns in data. Language data is rich in content, containing lexicon, syntax, semantics, pragmatics, and world knowledge. We can observe that LLMs have learned not only lower-order patterns related to lexicon and syntax, but also higher-order patterns related to semantics, pragmatics, and world knowledge. Previous language models often could not achieve this, and this is precisely the capability

that has "emerged" in ChatGPT and subsequent LLMs. Therefore, the view that LLMs merely learn the form of language without learning its content (e.g., Chomsky's view discussed later) is not convincing.

- **The basic implementation principle can be summarized as Next Token Prediction (NTP), but overall capabilities are jointly determined by strategy, model, algorithm, and data.** The learning and inference process of LLMs is NTP, but this is merely the surface form; the specific implementation methods and their characteristics are more important. Maximum likelihood estimation used in pre-training (equivalent to data compression) estimates the probability distribution of token sequence data. Reinforcement learning in post-training aims to fine-tune the model to become an optimal policy function for generating reasonable token sequences. The Transformer as a model has extremely strong language and knowledge representation capabilities. The stochastic gradient descent optimization algorithm can help find solutions with good generalization. The key to LLMs lies in the systematic integration and scaled implementation of these technologies. The view that attributes LLM success simply to NTP is an oversimplified understanding.
- **The internal mechanisms of LLMs have been analyzed and understood to a certain extent.** In recent years, LLM interpretability research has made some progress, and LLMs are no longer completely black boxes to us. Features in LLMs can be extracted using tools such as SAE, and circuits formed between features can be traced using tools such as CLT. ByteDance's recent work further reveals the patterns by which features in LLMs are memorized during learning and retrieved during inference. As future research continues to deepen, the mechanics of LLMs will be increasingly analyzed and understood.

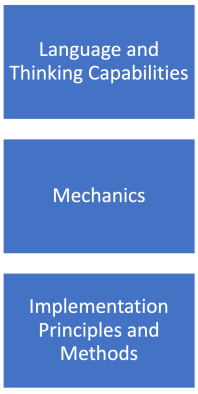


Figure 1: The relationship among LLM's language and thinking capabilities, mechanics, and implementation principles and methods.

2 Human Language Understanding and Thinking

We first summarize the characteristics of human language understanding and thinking.

2.1 Language Understanding

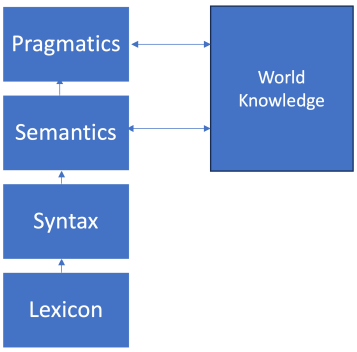


Figure 2: Human language understanding process

Language processing is divided into understanding and generation. Human language understanding can be defined from internal and external perspectives. The internal perspective refers to a person's understanding of language in the mind; the external perspective refers to observed language understanding by a third party.

Internal language understanding involves the representation of language in the human mind. This includes concept grounding, i.e., connecting heard or seen language with existing knowledge. For example, when a person hears "giant panda," they connect it with their knowledge about giant pandas.

External language understanding can often be viewed from the perspective of task completion, with goals and results. Dialogue, such as question answering, can also be viewed as a process of task completion. (The Turing test judges whether a computer can understand language from an external perspective.)

The embodied cognition hypothesis holds that human language understanding is achieved through generating mental imagery (Image) in consciousness. For example, when answering the question "Does a gorilla have a nose?" an image of a gorilla appears in the mind.

Language is formally sequential data—whether speech or text in images, it usually needs to be processed in order; but in the human brain, the understanding process involves hierarchical and parallel processing.

Language simultaneously contains lexicon, syntax, semantic, and pragmatic information, and semantics and pragmatics are closely related to world knowledge (see Figure 2). From the perspective of abstraction level, the former two

belong to low-level information, while the latter two belong to high-level information. In the above example, “gorilla,” “has,” and “nose” belong to semantic information, while the intent expressed by the entire interrogative sentence belongs to pragmatic information.



Figure 3: Chunk analysis example: phrases within boxes are chunks

Regardless of the language or how great the grammatical differences, languages can usually be viewed as composed of chunks at different granularities and can be analyzed from the chunk perspective (see Figure 3).

2.2 Thinking

During the process of human thinking, neurons in the brain’s neural system are activated, forming neural representations in the brain. At the same time, in the human mind, especially at the conscious level, symbolic representations or mental images are formed. The generation of symbolic representations is an important view of traditional cognitive science, while the generation of mental images is advocated by embodied cognition.

Embodied cognition holds that human reasoning relies heavily on analogy. For example, reasoning that “this phone is the same brand as the one I used before, so the quality should also be good.”

In rigorous mathematical derivations, logical inference rules (such as syllogisms) or computational rules (such as addition, subtraction, multiplication, and division rules) are primarily used. But these only exist in specific scenarios.

Bayesian inference can be used to characterize the human reasoning process. For example, the reasoning in Sherlock Holmes’s detective process can be viewed as Bayesian inference.

3 LLM Learning and Inference

An LLM is a type of language model whose processing form can be summarized by Next Token Prediction (NTP), but its overall mechanism is more complex and cannot be simply understood by this concept alone.

In the pre-training phase, maximum likelihood estimation is performed on token sequence data to obtain the optimal model for generating token sequences. In the post-training phase, reinforcement learning is used to further fine-tune the model, making it an optimal policy in the sense of generating reasonable token sequences; the implementation learns **token-level** policies, but can be approximately viewed as learning **sequence-level** policies.

In terms of model architecture, LLMs adopt autoregressive Transformer decoders; in terms of optimization algorithms, stochastic gradient descent (e.g., AdamW) is used.

The language capabilities of LLMs result from the joint action of model structure, optimization algorithm, training strategy, and language data characteristics. Pre-training determines the basic language and reasoning capabilities, while post-training further enhances reasoning capabilities and task completion abilities.

This section summarizes the basic principles and implementation methods of LLMs. The content involves technical details, and readers may skip it as appropriate.

3.1 Pre-training

In pre-training, the LLM represents the generation probability of token sequences. According to the multiplication theorem of probability, the probability of a token sequence $\boldsymbol{w}$ can be written as

$$P_{\boldsymbol{\theta}}(\boldsymbol{w}) = \prod_{t=1}^{T} P_{\boldsymbol{\theta}}(w_t|w_{<t}), \quad \boldsymbol{w} = w_1 w_2, \cdots, w_T \tag{1}$$

The learning objective is to estimate model parameters $\boldsymbol{\theta}$ for computing the conditional probability $P_{\boldsymbol{\theta}}(w_t|w_{<t})$. The prediction objective is to autoregressively generate token sequences based on the conditional probability $P_{\boldsymbol{\theta}}(w_t|w_{<t})$. LLMs typically use a Transformer decoder as the model to represent the conditional probability $P_{\boldsymbol{\theta}}(w_t|w_{<t})$.

Learning is conducted via maximum likelihood estimation, i.e., maximizing the likelihood function of the training data, or equivalently minimizing the cross-entropy loss. Estimating the model parameters is equivalent to the **modeling** step in lossless compression of training data. The cross-entropy loss is written as

$$L(\boldsymbol{\theta}) = \mathbb{E}_{w\sim P_{\text{data}}}\left[\sum_{t=1}^{T} -\log P_{\boldsymbol{\theta}}(w_t \mid w_{<t})\right] \tag{2}$$

Its gradient function is

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{w}\sim P_{\text{data}}}\left[\sum_{t=1}^{T} -\nabla_{\boldsymbol{\theta}} \log P_{\boldsymbol{\theta}}(w_t \mid w_{<t})\right] \tag{3}$$

Lossless compression consists of two steps: first estimating the model of the data distribution, and second encoding the data based on the model. Here only

the first step of modeling is performed, without the second step of encoding. Ilya Sutskever's proposed view of "compression is intelligence" can serve as an interpretation of LLM pre-training modeling.

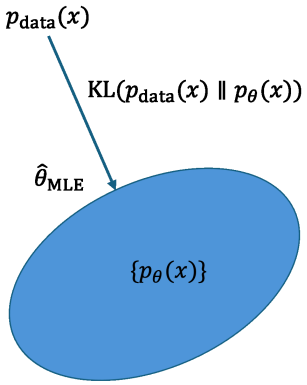


Figure 4: The distribution obtained by maximum likelihood estimation converges in probability to the KL projection of the true distribution onto the parametric family of distributions.

Maximum likelihood estimation possesses excellent properties under asymptotic conditions, i.e., when the data size tends to infinity. Consider the probability distribution space; the learning objective is to find the optimal distribution within the parametric family $\{p_{\boldsymbol{\theta}}(\boldsymbol{x})\}$. The parametric family forms a manifold in the space. Assume that the true data distribution $p_{\text{data}}(\boldsymbol{x})$ lies outside the manifold, as shown in Figure 4. Under certain regularity conditions, the distribution represented by the MLE parameter $\hat{\boldsymbol{\theta}}_{\text{MLE}}$ converges in probability to the KL projection of the true distribution onto the manifold, i.e., the distribution with the smallest divergence from the true distribution in terms of KL divergence.

$$\hat{\boldsymbol{\theta}}_{\text{MLE}} = \arg\min_{\boldsymbol{\theta}\in\boldsymbol{\Theta}} \text{KL}(p_{\text{data}}(\boldsymbol{x}) \,\|\, p_{\boldsymbol{\theta}}(\boldsymbol{x})) \tag{4}$$

If the true distribution exists within the parametric family, then the distribution represented by the MLE parameter $\hat{\boldsymbol{\theta}}_{\text{MLE}}$ converges in probability to the true distribution.

3.2 Post-training

LLM post-training includes two phases: SFT (Supervised Fine-Tuning) and RL (Reinforcement Learning). A special case of the latter is RLHF (Reinforcement Learning from Human Feedback).

In post-training RL, the LLM is the policy function, where the state is the token sequence generated so far, and the action is the next token. The reward is generally defined at the last position of the token sequence, representing the reasonableness of the sequence, with rewards at other positions being zero.

Post-training RL typically adopts policy gradient methods, using the policy function for rollout of action trajectories, i.e., token sequences. SFT can be viewed as imitation learning, with the objective of fitting given demonstration data.

The learning objective is to maximize the return, i.e., the expected reward, to fine-tune the LLM's model parameters $\boldsymbol{\theta}$. The policy function $\pi_{\boldsymbol{\theta}}(y_t|\boldsymbol{x}, y_{<t})$ represents the generation probability of the next token y_t given the prompt $\boldsymbol{x}$ and the already-generated prefix $y_{<t}$. In the simplest REINFORCE algorithm, the gradient of the return is written as

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{x}\sim P_{\text{data}}, \boldsymbol{y}\sim\pi_{\boldsymbol{\theta}}}\left[R(\boldsymbol{x}, \boldsymbol{y}) \sum_{t=1}^{T} \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(y_t \mid \boldsymbol{x}, y_{<t})\right] \tag{5}$$

where $\boldsymbol{x}$ denotes the prompt token sequence, $\boldsymbol{y}$ denotes the response token sequence, and $R(\boldsymbol{x}, \boldsymbol{y})$ denotes the reward. In practical methods (such as PPO, GRPO), advantage functions and regularization terms are introduced to stabilize training, but all belong to policy gradient methods.

From the form of the generation process (Equations (3) and (5)), the post-training phase still generates tokens one by one, which shares similarity with pre-training. The difference is that the post-training objective is no longer to learn the data distribution, but to learn to generate sequences that maximize the expected (average) reward.

In fact, post-training RL can be defined at two levels: **token-level and sequence-level**. Current RL implementations are all at the token level, as described above. However, from the perspective of conceptual understanding, sequence-level RL is more intuitive. Token-level RL is usually an approximation of sequence-level RL [9].

The objective of sequence-level RL is to learn how to generate the response $\boldsymbol{y}$ with maximum expected reward given the prompt $\boldsymbol{x}$. Its gradient can be expressed as:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\boldsymbol{x}\sim P_{\text{data}}, \boldsymbol{y}\sim\pi_{\boldsymbol{\theta}}}\left[R(\boldsymbol{x}, \boldsymbol{y}) \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(\boldsymbol{y}|\boldsymbol{x})\right] \tag{6}$$

where the reward function $R(\boldsymbol{x}, \boldsymbol{y})$ evaluates the reasonableness of the response, and the policy $\pi_{\boldsymbol{\theta}}$ generates complete sequences token by token via autoregression. In this simple case, token-level and sequence-level RL are equivalent.

3.3 Inference

LLM reasoning can be viewed as search in a vast space composed of token sequences. We call these token sequences "statements." Statements express certain content and serve as the basic units of reasoning. They can be natural language (such as a passage of text) or formal language (such as a piece of code); they can express rigorous logical reasoning or analogical reasoning; a chain of thought may contain multiple statements.

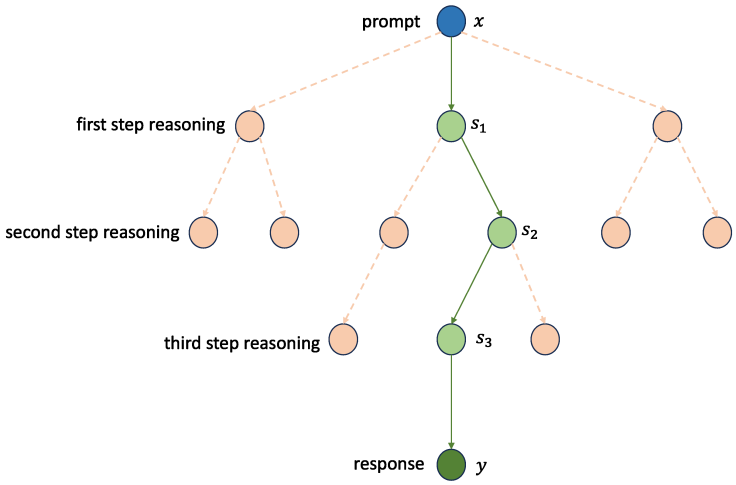


Figure 5: Illustration of LLM reasoning as search in the statement space

During the search process, the LLM continually selects appropriate reasoning paths and ultimately obtains an answer (see Figure 5). After RL training, model parameters remain fixed during the inference phase, but by generating longer chains of thought, sampling more paths, etc., more thorough search can be conducted in the statement space to find better answers. This process enables Test-Time Scaling.

RL training can enhance the model's reasoning capability. By optimizing the policy, the model becomes more inclined to generate high-quality intermediate and final results, essentially enhancing search capability [6].

Formalizing reasoning as search in the statement space enables LLMs to transcend mere language understanding and generation, moving toward higher-level intelligence with "thinking" capabilities.

3.4 Optimization Algorithm: Stochastic Gradient Descent

Once the learning strategy is determined, the next step is to use optimization algorithms to estimate optimal parameters and optimize the objective function. As long as the loss function of a neural network is differentiable with respect to parameters, it can be optimized by stochastic gradient descent (SGD) and its variants. This is also the case for LLMs.

Experiments have shown that using SGD and its variants (such as AdamW) not only allows the loss function to converge to good minima regions, but the learned models typically possess good generalization ability. (See Figure 6)

SGD tends to converge to relatively flat minima regions, which plays an important role in deep learning generalization performance. Moreover, SGD usually achieves better generalization when the batch size is smaller [5].

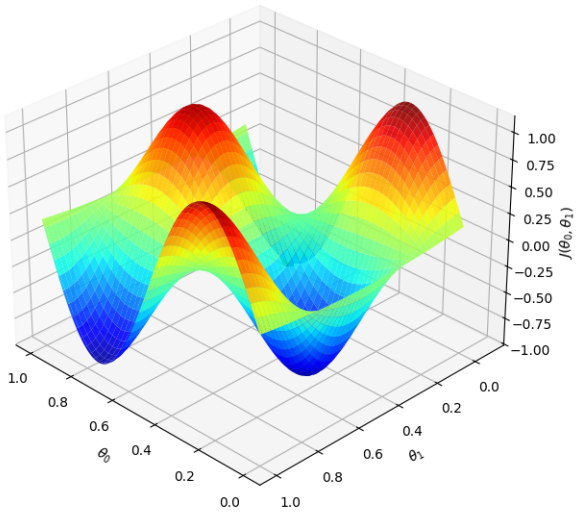


Figure 6: Stochastic gradient descent

Experiments have shown that deep learning models are often over-parameterized, meaning that model capacity far exceeds data scale. This differs from traditional machine learning. The training loss function exhibits a double descent phenomenon [1] (Figure 7 is an illustration). In the under-parameterized region, as model parameters increase, training loss decreases while test loss increases—traditional machine learning operates in this region. After training loss approaches zero (past the interpolation threshold, i.e., the critical point where the model completely fits the training data), the model enters the over-parameterized region, where as model capacity continues to increase, training loss remains near zero while test loss also decreases—deep learning operates in this region [7].

Empirically, LLM pre-training also operates in the over-parameterized region, so SGD can achieve very good generalization, with test loss following the Scaling Law [4]. The Scaling Law refers to the regularity that loss decreases as a power law with compute, parameter count, and data size. Similar trends have been observed in post-training (RL), but the mechanisms differ and require further study.

3.5 Model: Transformer

LLMs typically use the Transformer decoder architecture, which plays an important role in language and knowledge representation. Particularly important are the self-attention, feed-forward network (FFN), and residual connections in the Transformer.

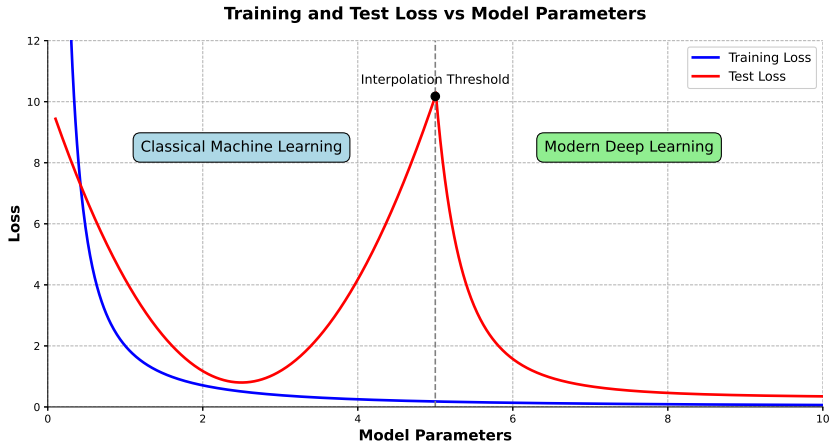


Figure 7: Training and prediction loss curves

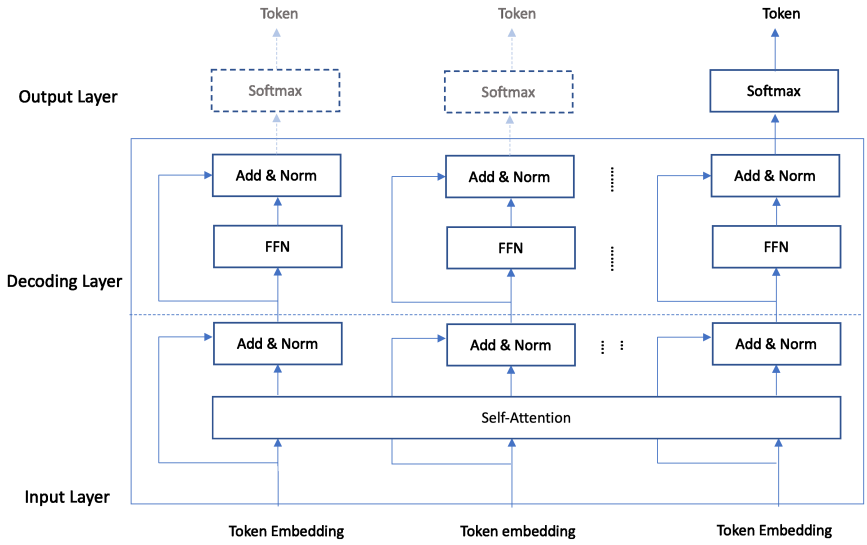


Figure 8: Transformer decoder architecture

Figure 8 shows the architecture diagram of the Transformer decoder. Table 1 lists the model scales of representative LLMs.

Self-Attention

Each layer of the Transformer has a self-attention module. The main role of self-attention is to linearly combine the input representation vector with the

Model	Data	Params	Layers	Context	Vocab	Dim
GPT-1	1B Tokens	117M	12	512	40k	768
GPT-2	10B Tokens	1.5B	48	1024	∼50k	1600
GPT-3	300B Tokens	175B	96	2048	∼50k	12288

Table 1: GPT model scale, training data, context window, and vocabulary size

context representation vectors to produce a new output representation vector.

First, linear transformations are applied to the input representation vector at the current position and the input vectors of its context, yielding the query matrix, key matrix, and value matrix. The attention computation is as follows:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V \tag{7}$$

where Q (Query), K (Key), and V (Value) are the query matrix, key matrix, and value matrix obtained by linear transformation of input tokens, respectively; d is the dimension of the key vector; and $\sqrt{d}$ is the scaling factor. Finally, a linear transformation is applied to the result to obtain the output vector at the current position. To ensure that the vector at the current position is only linearly combined with vectors at previous positions, masking is required in the matrix computation.

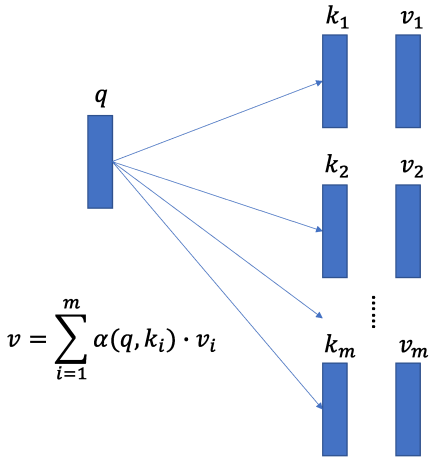


Figure 9: Illustration of self-attention computation

Intuitively, self-attention can be understood as a key-value store query: for each query vector, compute its similarity with all key vectors, and take the weighted sum of all value vectors using the similarities as weights as the result. The more a query matches a key, the greater the weight of the corresponding

value in the final result. It is a "soft" key-value store query; in the extreme case where all vectors are one-hot vectors, self-attention computation is equivalent to symbol-based key-value store query. Figure 9 shows an example of self-attention computation.

The above describes single-head self-attention; in practice, multi-head self-attention is used, computing attention in parallel across different subspaces, then concatenating and applying a linear transformation. Different attention heads can focus on different types of compositional relationships—for example, some heads may focus on grammar, some on semantics, and some on pragmatics. Through the multi-head mechanism, the model can capture compositional relationships between representation vectors from multiple perspectives.

Self-attention is the composition of representation vectors. Related knowledge is also stored in the linear transformation matrices. One of the most essential features of language is compositionality: words compose into phrases, phrases into sentences, and sentences into discourse. The self-attention mechanism directly supports this compositionality. The attention computation at each layer recombines the input sequence, and the accumulation across multiple layers achieves progressive composition from local to global, from shallow to deep.

Feed-Forward Neural Network (FFN)

Each layer of the Transformer contains a two-layer feed-forward neural network (FFN) module.

$$\boldsymbol{y} = \boldsymbol{W}_2 \mathrm{ReLU}(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2 \tag{8}$$

where $\boldsymbol{x}$ is the input vector, $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$ are weight matrices, $\boldsymbol{b}_1$ and $\boldsymbol{b}_2$ are biases, ReLU is the activation function, and $\boldsymbol{y}$ is the output vector. Figure 10 shows a simple example of a two-layer FFN.

The main role of the FFN is to perform feature detection on the input vector and represent the detected features as the output vector. Related knowledge is also stored in the neural network's weight matrices.

Each neuron in the FFN's intermediate layer can be viewed as detecting features in the input vector. Specifically, the input vector $\boldsymbol{x}$ undergoes a dot product with the weight vector (row vector of W_1). If the input direction aligns with the weight direction, the dot product is positive, and the feature represented is activated after ReLU; if the directions are opposite, the dot product is negative, and the feature is not activated after ReLU; this produces the intermediate vector. Output layer neurons are linear neurons that transform the intermediate vector into the output vector through linear transformation (using W_2). The input and output vectors have the same dimension, while the intermediate vector's dimension (i.e., the number of intermediate layer neurons) is typically about 4 times larger, enabling more feature detection.

The FFN serves as a feature detector, and the training process teaches the weight matrices effective feature detection patterns. The corresponding knowledge is stored in the weight matrices. In recent years, interpretability research

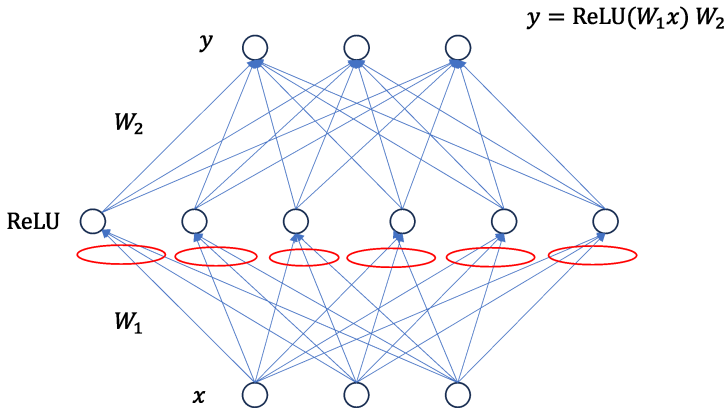


Figure 10: Illustration of a two-layer feed-forward neural network structure

has shown that neuron representation of features is not "one-to-one," but exhibits superposition phenomena. This will be discussed in detail in Section 4.2.

The FFN layers at different depths of the Transformer store different types of knowledge. Research has found that: shallow layers primarily represent lexical and grammatical information; middle layers capture more semantic information; and deep layers handle information potentially related to context, pragmatics, and reasoning.

Modern LLMs are very large in scale (see Table 1), and parameter expansion enables them to accommodate vast amounts of knowledge. In particular, introducing Mixture of Experts (MoE) structures in FFN layers can dramatically increase model capacity, enabling the representation and storage of richer knowledge.

Residual Connections

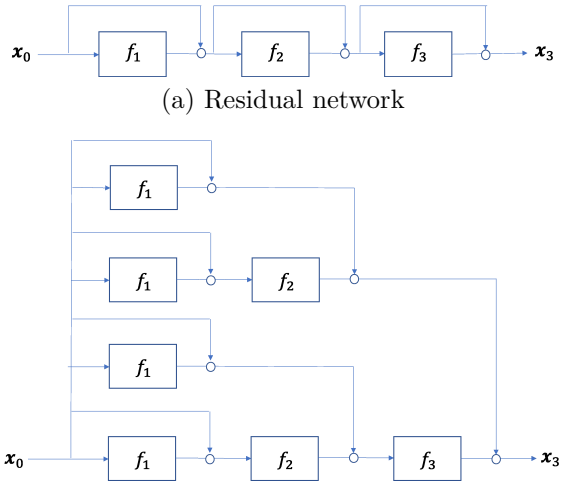
The Transformer has residual connections after both the self-attention module and the FFN module at each layer.

$$\boldsymbol{x}_{l+1} = \boldsymbol{x}_l + f(\boldsymbol{x}_l) \tag{9}$$

where f represents self-attention or FFN. Based on the LLM's residual structure, representations learned at different layers can be accumulated and reused in deeper layers through additive pathways.

LLMs can be understood as implicit ensembles of sub-networks at different depths formed by self-attention and FFN. As shown in Figure 11, suppose three functions are connected in series; if residual connections are added between them, the entire network can be interpreted as an implicit ensemble composed of sub-networks at different depths.

Residual connections serve several functions: they facilitate cross-layer propagation of information and gradients, allow shallow representations to directly



(a) Residual network

(b) Equivalent implicit ensemble network

Figure 11: Example of residual neural network

influence deeper layers, and improve the trainability of deep networks.

Residual connections add further difficulty to interpretability analysis of LLMs. While the self-attention weights of LLMs can be used to analyze correlational relationships between tokens at different positions, because information can propagate between layers through residual connections, these correlational relationships do not necessarily originate solely from the current layer.

Other Mechanisms

LLMs explicitly represent the sequential position of tokens through positional encoding (such as RoPE), which is significantly different from RNNs (Recurrent Neural Networks) that implicitly represent positional information through model structure. Layer normalization (typically in Pre-Norm form) is one of the important mechanisms that enable stable Transformer training.

Furthermore, while maintaining the basic Transformer architecture, current LLMs have introduced several key improvement techniques. For example: Mixture of Experts (MoE) for improving model capacity and computational efficiency; Swish activation function replacing ReLU; SwiGLU structure improving FFN modules; Multi-Query Attention (MQA) or Grouped-Query Attention (GQA) for improving attention computation efficiency. These improvements primarily occur at the module design and computational implementation level, without fundamental changes to the overall architecture.

4 Mechanics of LLMs

LLM research can be conducted from three perspectives: machine learning methods and theory, external prompt experiment analysis, and internal mechanics research. If we compare LLMs to the human brain, mechanics research corresponds to neuroscience experiments.

4.1 Feature Superposition

"Feature superposition" phenomena may exist at each layer of neural networks. The traditional view holds that one neuron represents one feature. However, extensive experiments show that this idealized situation is relatively rare in actual networks. Instead, the relationship between neurons and features often exhibits a many-to-many correspondence: one neuron participates in representing multiple features, and one feature is jointly represented by multiple neurons.

The Anthropic research team proposed the Superposition Hypothesis [2]. Its core idea is: **through feature superposition, one layer of neurons in a neural network can approximately represent far more features than the number of neurons**, at the cost of some degree of interference between features.

A layer of the neural network (called the actual layer) can be represented as:

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}) \tag{10}$$

where $\boldsymbol{x}$ is the input vector in the input space; $\boldsymbol{W}$ is the weight matrix; $\boldsymbol{b}$ is the bias vector; $\boldsymbol{h}$ is the output vector or feature vector; and ReLU is the activation function.

Feature superposition theory posits that a wider hypothetical neural network layer exists, using more neurons to explicitly represent a large number of features:

$$\bar{\boldsymbol{h}} = \mathrm{ReLU}(\bar{\boldsymbol{W}}\boldsymbol{x} + \bar{\boldsymbol{b}}) \tag{11}$$

where $\bar{\boldsymbol{h}}$ is the feature vector, with each dimension corresponding to a feature, satisfying $\dim(\bar{\boldsymbol{h}}) \gg \dim(\boldsymbol{h})$, i.e., the dimension of the wider layer far exceeds that of the actual layer; $\bar{\boldsymbol{W}}$ and $\bar{\boldsymbol{b}}$ are the weight matrix and bias vector.

Importantly, the feature vector $\bar{\boldsymbol{h}}$ of the wider layer is sparse, while the feature vector $\boldsymbol{h}$ of the actual layer is dense. Sparsity means that for each input, only a few features are activated (e.g., only a few dozen out of ten thousand features are activated), keeping interference between different features small.

The superposition hypothesis holds that an approximate equivalence relationship exists between the wider layer and the actual layer. Specifically, both can approximately reconstruct the input vector $\boldsymbol{x}$ through linear transformation, yielding an approximate reconstruction $\hat{\boldsymbol{x}}$, as shown in Figure 12. Therefore, it can be considered that **in the actual network, the sparse feature vector $\bar{\boldsymbol{h}}$ is compressed into the dense feature vector $\boldsymbol{h}$. The sparse feature vector $\bar{\boldsymbol{h}}$ is nearly mutually independent (non-superposed), while the dense feature vector $\boldsymbol{h}$ is superposed**.

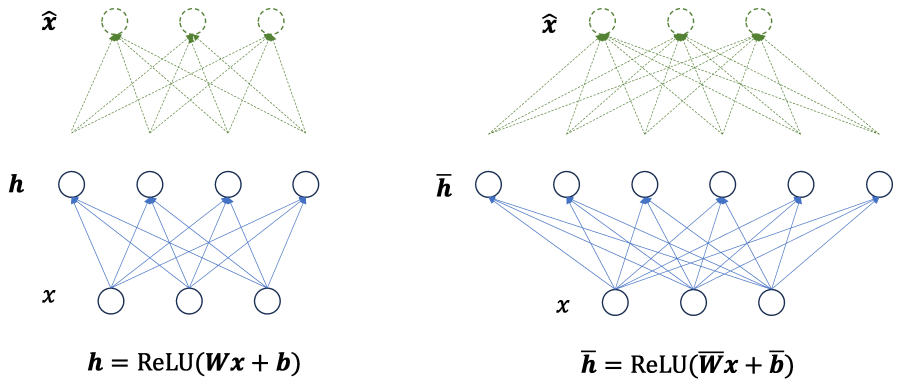


Figure 12: The original feed-forward neural network and an approximately equivalent wider neural network

High-dimensional geometry theory provides certain guarantees for the existence of the wider layer. Under relevant definitional conditions, the number of nearly orthogonal basis vectors in an n-dimensional space can reach an exponential level in n. Suppose $\bar{\boldsymbol{v}}_1, \bar{\boldsymbol{v}}_2, \ldots, \bar{\boldsymbol{v}}_m$ are a set of nearly orthogonal basis vectors in the input space; then the input vector can be approximately decomposed as:

$$\boldsymbol{x} \approx \sum_{i=1}^{m} \bar{h}_i \bar{\boldsymbol{v}}_i, \quad \bar{\boldsymbol{h}} = (\bar{h}_1, \bar{h}_2, \cdots, \bar{h}_m) \tag{12}$$

Here each basis vector $\bar{\boldsymbol{v}}_i$ and its activation value $\bar{h}_i$ correspond to a feature. When basis vectors are nearly orthogonal and the feature vector is sparse, the input vector can be well represented while reducing interference between features.

The superposition hypothesis posits that deep learning methods can achieve this effect, i.e., the actual layer (dense feature vector) implements compression of the wider layer (sparse feature vector), or equivalently, the actual layer (dense feature vector) implicitly contains the wider layer (sparse feature vector). There are primarily two reasons for this.

First, the features contained in a general input vector $\boldsymbol{x}$ are sparse in number. For example, when processing the token "bridge" in "I visited the Golden Gate Bridge," the wider layer feature vector $\bar{\boldsymbol{h}}$ may have only a few features activated (such as "Golden Gate Bridge," "San Francisco," "bridge structure," "tourist attraction," etc.), with all other feature values being zero. This sparsity ensures that interference between the large number of nearly orthogonal features is sufficiently small, making the superposition mechanism practically feasible.

Second, during training, neural networks minimize loss functions through gradient descent. When the network faces the two objectives of "representing as many features as possible" and "using as few neurons as possible," feature superposition becomes a natural optimization outcome. Additionally, the use

of ReLU activation functions also promotes sparsification of feature vectors, as weak activation values are truncated to zero.

The reasonableness of the above superposition hypothesis has been verified in Anthropic's toy model simulation experiments, and has received further support in the subsequent development and application of Sparse Autoencoders.

4.2 SAE: Feature Analysis

Sparse Autoencoders (SAE) can be used to analyze neural networks and discover interpretable features within them. In LLM interpretability research, they are typically applied to the Transformer's residual stream, i.e., to the output representation vectors at each layer.

SAE and feature superposition theory form a complementary relationship. Feature superposition can be viewed as a compression process: the model implicitly represents input vectors through high-dimensional and sparse feature vectors. SAE can be viewed as a "decompression" method: decomposing input vectors into high-dimensional and sparse feature vectors. This "compression–decompression" relationship makes SAE an important tool for studying and analyzing feature superposition phenomena.

SAE consists of an encoder and a decoder. First, the encoder transforms the input vector into a high-dimensional and sparse feature vector through nonlinear transformation:

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{W}_e \boldsymbol{x} + \boldsymbol{b}_e) \tag{13}$$

where $\boldsymbol{x} \in \mathbb{R}^n$ is the residual stream from a certain layer of the LLM, $\boldsymbol{W}_e$ is the encoder weight matrix, $\boldsymbol{b}_e$ is the bias vector, $\boldsymbol{h} \in \mathbb{R}^m$ is the feature vector, satisfying $m \gg n$, i.e., the feature dimension is far greater than the input dimension.

The decoder reconstructs the original input vector from the feature vector through linear transformation:

$$\hat{\boldsymbol{x}} = \boldsymbol{W}_d \boldsymbol{h} + \boldsymbol{b}_d \tag{14}$$

where $\boldsymbol{W}_d$ is the decoder weight matrix and $\boldsymbol{b}_d$ is the bias vector. The decoder uses linear transformation (without activation function), and this design is consistent with the linear combination assumption of features in superposition theory.

SAE training balances two objectives: on one hand, making the reconstructed vector $\hat{\boldsymbol{x}}$ as close as possible to the original input vector $\boldsymbol{x}$; on the other hand, introducing L_1 or approximate L_0 regularization to promote sparsity of the feature vector $\boldsymbol{h}$.

In practice, SAE training requires extracting large amounts of data from the target LLM. Large-scale corpora are fed into the LLM, and the activation vectors produced by the model when processing each token (such as the residual stream at each layer) are collected. These vectors $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$ constitute the SAE training dataset. After training, for any input $\boldsymbol{x}$, the encoder output $\boldsymbol{h}$ typically exhibits strong sparsity.

The sparse activation features obtained through SAE are consistent with the predictions of superposition theory, i.e., the model may encode far more latent concepts in its neurons than the number of neurons. For example, researchers have successfully extracted features on the order of hundreds of thousands to millions when analyzing large language models, some of which have clear semantic meanings, such as features related to entities (like "Golden Gate Bridge") or behaviors (like "sycophancy").

Analysis shows that features in large language models often exhibit a certain hierarchical structure: shallow layers primarily represent lexical and simple grammatical features of the input; middle layers contain many complex grammatical and basic semantic features; and deep layers primarily handle complex semantic, reasoning implementation, and output expression features.

4.3 Memory Mechanism

ByteDance's work proposed the Function Token Hypothesis, revealing the basic characteristics of LLM memory mechanisms [8]. The Function Token Hypothesis posits that the memorization of features in LLMs is organized around function tokens, and the retrieval of features in a given context is also conducted through function tokens.

Function tokens are the tokens that appear most frequently in the training corpus. Most of them correspond to function words in linguistics, playing crucial roles in grammar and contextual connections. For example, the article "the," punctuation marks (commas, periods), newline characters, etc. In contrast, content tokens express clear and rich semantic information. Statistics show that in large-scale pre-training corpora, the top 100-plus high-frequency tokens account for approximately 40% of all token occurrences.

During the pre-training phase of LLMs, the learning process exhibits prominent characteristics centered around function tokens. By decomposing the training loss according to the four combinations of function tokens and content tokens, results show that the loss for "function token $\rightarrow$ content token" decreases the slowest. That is, predicting the next content token given a function token is the most difficult. From a linguistic perspective, this is reasonable, because function tokens often mark the end of the previous linguistic unit (chunk), and predicting the content token after it requires accurate understanding of the entire context from the beginning to the current position. It can be inferred that this most difficult prediction task becomes the dominant force driving model optimization.

Another finding is that function tokens can activate most features during training (activating different sparse features in different contexts). A bipartite graph is established between function tokens and features. If a function token activates a certain feature in a certain context, an edge is established between the two. As training progresses, edges on the bipartite graph continuously increase. Eventually, a small number of function tokens can establish connections with most features—the top 10 high-frequency tokens activate 70% of features, meaning these function tokens can (in different contexts) activate most features.

Power-law distributions also exist here.

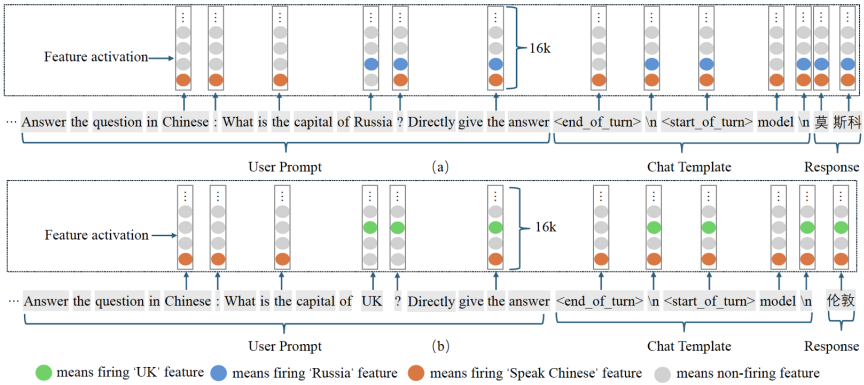


Figure 13: Function tokens play a core role in memory retrieval during LLM inference.

During inference, function tokens play a core role in memory retrieval. They can dynamically activate the most predictive features from context, thereby guiding the generation of the next token. For example, as shown in Figure 13, when the prompt is "Answer the question in Chinese: What is the capital of Russia?", function tokens (such as the colon ":" and newline character) activate features in the context such as "answer in Chinese" and "Russia," while suppressing irrelevant features, ultimately guiding the model to generate the answer "Moscow" in Chinese. This dynamic feature selection and combination capability is precisely what distinguishes function tokens from content tokens.

The reason function tokens play such a critical role in LLMs is the joint result of training objectives, learning algorithms, model architecture, and language characteristics. First, the next-token prediction training objective (cross-entropy loss) requires the model to maximize prediction accuracy, and the gradient descent algorithm always prioritizes reducing the largest losses. Second, the feed-forward network layers in the Transformer architecture can represent and memorize knowledge (features) well, while self-attention layers can effectively combine lower-order knowledge (features) into higher-order knowledge (features). Finally, the structural characteristics of natural language itself play a decisive role—text is always segmented by function tokens into nested chunks (which can be phrases, sentences, or paragraphs). Therefore, prediction after function tokens requires understanding the semantics of the entire context from the beginning of the text to that position. This is an extremely challenging task that drives function tokens to acquire the ability to connect with most features during training and reactivate the most predictive features during inference.

The Function Token Hypothesis has profound implications for LLM training practice. The most important point is that the format of training data is crucial. Results from multiple studies corroborate this point. In the post-training phase,

only a small number of training steps can significantly improve the model's instruction-following, chain-of-thought reasoning, and other capabilities. This may be because post-training activates features already acquired during pre-training by adjusting the activation patterns of function tokens. For example, function tokens (such as "thus") can significantly improve reasoning performance in reinforcement learning training.

4.4 CLT: Circuit Analysis

Circuits refer to computational graphs that connect features across layers in LLMs, used to represent how features in the model are activated and propagated. Because SAE can only see features at a single layer, it has significant limitations. To analyze cross-layer feature connections and influence relationships, researchers proposed the CLT (Cross Layer Transcoder) method.

The working principle of CLT is: taking the residual stream $\boldsymbol{x}^{(l)}$ at a certain layer as input, the model maps it to the residual streams at subsequent layers $\boldsymbol{x}^{(l+1)}, \boldsymbol{x}^{(l+2)}, \ldots, \boldsymbol{x}^{(L)}$. In this way, CLT can learn a cross-layer aligned feature dictionary, capturing the influence relationships between features at different layers.

CLT has a feature extraction module similar to SAE at each layer, but its optimization objective is quite different. The output at each layer is the reproduced residual streams of subsequent layers. It consists of nonlinear transformation (corresponding to the encoder), linear transformation (corresponding to cross-layer mapping), and linear decoder transformation:

$$\boldsymbol{h}^{(l)} = \mathrm{ReLU}(\boldsymbol{W}_e^{(l)} \boldsymbol{x}^{(l)} + \boldsymbol{b}_e^{(l)}) \tag{15}$$

$$\boldsymbol{h}^{(l+k)} = \boldsymbol{T}^{(l \to l+k)} \boldsymbol{h}^{(l)} + \boldsymbol{b}_T^{(l+k)} \tag{16}$$

$$\hat{\boldsymbol{x}}^{(l+k)} = \boldsymbol{W}_d^{(l+k)} \boldsymbol{h}^{(l+k)} + \boldsymbol{b}_d^{(l+k)} \tag{17}$$

where $\boldsymbol{W}_e^{(l)}$ is the encoder weight matrix at layer l, $\boldsymbol{b}_e^{(l)}$ is the bias vector; $\boldsymbol{W}_d^{(l+k)}$ is the decoder weight matrix at layer $l+k$, $\boldsymbol{b}_d^{(l+k)}$ is the bias vector; $\boldsymbol{T}^{(l \to l+k)}$ is the cross-layer linear transformation weight matrix that maps features from layer l to layer $l+k$, and $\boldsymbol{b}_T^{(l+k)}$ is the bias vector. Through cross-layer linear transformation, $\boldsymbol{T}^{(l \to l+k)}$ implements a linear projection from the feature space at layer l to the feature space at layer $l+k$. The training objective of CLT consists of minimizing the reconstruction error across all layers and sparsity regularization.

Based on CLT analysis results, attribution graphs can be constructed, intuitively displaying the linear mapping relationships of features between model layers, helping to understand the knowledge representation and computational mechanisms inside LLMs.

The construction of attribution graphs is based on trained CLT models. Given a specific input prompt, the original Transformer model is first run, recording the residual streams $\boldsymbol{x}^{(l)}$ at each layer. Then, CLT is used to extract

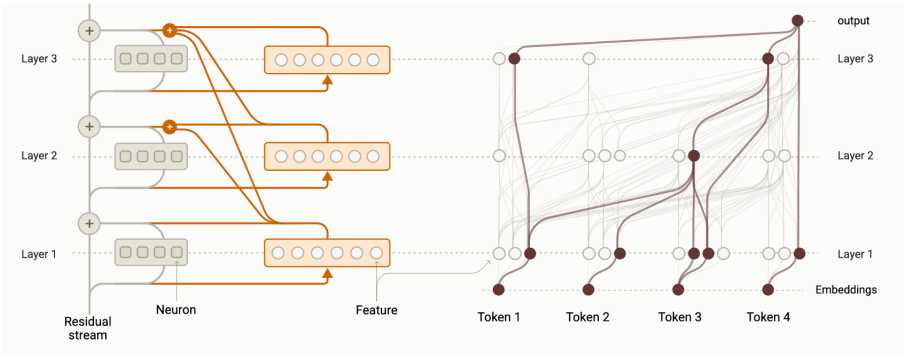


Figure 14: Attribution graph constructed based on CLT, used for analyzing the internal computational mechanisms of LLMs.

sparse features at each layer $\boldsymbol{h}^{(l)} = \text{ReLU}(\boldsymbol{W}_e^{(l)}\boldsymbol{x}^{(l)} + \boldsymbol{b}_e^{(l)})$, and the mapping relationships between features are analyzed through cross-layer mapping matrices $\boldsymbol{T}^{(l \to l+k)}$.

The attribution graph is a directed acyclic graph representing the computational paths from input tokens to output tokens. In this graph, nodes represent activated features or token embeddings in the model, while edges represent influence relationships between nodes. Specifically, if feature i is activated at layer l ($h_i^{(l)} > 0$) and has a significant contribution to the activation of feature j at layer $l + k$ through cross-layer mapping $\boldsymbol{T}^{(l \to l+k)}$, then a directed edge from node (i, l) to $(j, l + k)$ is added to the graph.

To improve the interpretability of attribution graphs, pruning techniques are used to simplify the graph. By setting thresholds, only nodes and edges with significant activation strength and contribution are retained. Gradient backpropagation is further used to identify paths with the greatest contribution to the final output. The resulting simplified attribution graph can more clearly display feature activation and reasoning paths in the model, revealing the core feature circuits in specific scenarios.

5 Language Understanding and Reasoning of LLMs

5.1 Capabilities of LLMs

5.1.1 Higher-Order Patterns

From their behavioral performance, large language models have demonstrated language and reasoning capabilities equal to or exceeding those of humans. Using the Turing test as a benchmark—i.e., examining whether they are indistinguishable from humans in dialogue—LLMs have reached human level.

What LLMs have learned is not limited to lower-order patterns of language, but also encompasses higher-order patterns of language and reasoning. This

can be intuitively verified in our daily use of LLMs. For example, LLMs can understand and execute instructions like "How tall is the Himalayas? Answer in English," demonstrating pragmatic capability; they can also analyze questions involving conceptual similarities and differences such as "the relationship between the Golden Gate Bridge and the Golden Arches," showing the integration of semantic understanding and world knowledge.

From the perspective of internal mechanism analysis, the existence of conceptual features such as "Golden Gate Bridge" and "sycophancy" in the model similarly reveals that LLMs possess understanding capabilities at the semantic and pragmatic levels.

Chomsky once criticized LLMs for merely learning surface-level statistical regularities of language. However, the above facts show that this judgment does not hold. Nevertheless, this does not mean that LLM language capabilities are completely equivalent to human language capabilities. In fact, the language mechanisms of LLMs differ significantly from the human brain. For example, human language understanding relies on the coordinated work of two brain regions—Broca's area and Wernicke's area: the former is primarily responsible for grammatical processing, while the latter handles lexical processing.

5.1.2 Overall Mechanism

The overall mechanics of large language models (LLMs) can be understood from the perspectives of training method, strategy, algorithm, and model (see Figure 15). Their training method typically includes pre-training and post-training phases, and through two-stage training integration, the model simultaneously possesses language understanding, generation, and reasoning capabilities within a unified system. Mechanistically, LLMs are based on autoregressive prediction—predicting the next token step by step based on existing context—which can also be viewed as a sequential decision-making process.

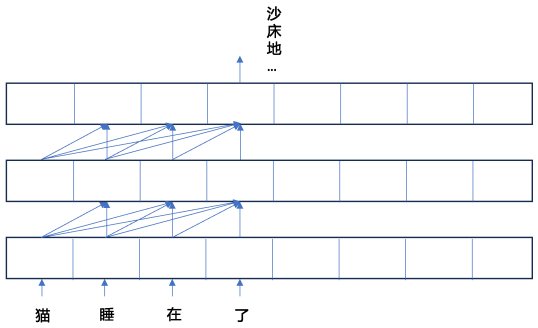


Figure 15: LLM mechanics can be understood from the perspectives of training method, strategy, algorithm, and model.

In the pre-training phase, the model learns statistical regularities through large-scale corpora—given sufficiently long preceding context, the probability

distribution of the next token tends to become more concentrated; in the post-training phase, through human feedback or policy optimization, the model is made more inclined to generate the most reasonable output that best matches human preferences given the same context.

From a technical implementation perspective, the learning strategy defines the prediction objective and optimization direction, the algorithm adjusts model parameters to achieve the optimal objective, and the model structure determines expressive power—achieving feature composition through self-attention, feature detection (nonlinear transformation) through feed-forward networks, and hierarchical representation through multi-layer structure. All of these play important roles in LLM's human-like language and reasoning capabilities.

Performance improvement also exhibits clear scaling effects: as data volume, parameter scale, and computational resources increase, model capabilities can undergo qualitative leaps. At the same time, data quality and training design (such as high-quality corpora and system prompt design) are also critical to effectiveness.

The LLM trained in this way forms a large number of features internally, representing various concepts. Depending on different contexts, different features are activated, dynamically forming circuits that implement complex language processing and reasoning mechanisms.

5.2 Comparison with Human Capabilities

Table 2 compares the capabilities of LLMs and humans. It can be seen that LLMs have achieved levels comparable to or exceeding humans in language and reasoning tasks. However, in other capability dimensions, not only may the mechanisms differ fundamentally, but their performance is not simply comparable. The following provides brief explanations and discussions.

Hallucination essentially originates from incorrect judgments about facts. LLMs cannot solve the hallucination problem on their own, because what they learn are statistical regularities in language data. Theoretical proofs show that under certain assumptions, the language generation process will necessarily produce hallucinations with a certain probability [3]. The hallucination problem can be mitigated through other mechanisms, such as Retrieval-Augmented Generation (RAG).

Human thinking encompasses multiple aspects, involving not only language, reasoning, and mathematics, but also the five senses (vision, hearing, touch, smell, taste) and the motor system. The embodied cognition hypothesis holds that the thinking process is not purely symbolic computation, but simulation in the mind based on bodily sensory-motor experiences. Current Multimodal Large Language Models (MLLMs) can associate language reasoning with visual, auditory, and other information, but their reasoning process typically occurs in language representation space. This processing approach fundamentally differs from the rich, conscious thinking humans engage in based on embodied experiences. Recently, some work has attempted multimodal reasoning, enabling models to "see and think" simultaneously; however, these methods are still rel-

Table 2: Comparison of LLM and Human Capabilities

Capability Dimension	**LLM**
Language capability	**Reaches or exceeds human level**
Reasoning capability	**Reaches or exceeds human level**
Factual cognitive capability	Has hallucinations
Multimodal processing capability	Not inherent, but can be extended to multimodal models
Mathematical capability	Has certain capability but with limitations
Thinking capability	Does not have the mechanism combining language, reasoning, and multimodal processing like humans
Emotion	Has emotional elements in language and reasoning, but mechanism differs from humans
Creativity	Has a certain degree of creativity
Consciousness	Different mechanism

atively preliminary explorations. Therefore, from the perspective of embodied cognition, the thinking capability of MLLMs still has a significant gap compared to humans.

LLMs neither reason based on formal logical rules (such as propositional logic) nor perform arithmetic operations according to computational rules. They can simulate through their generation mechanism, exhibiting certain heuristic reasoning and computational capabilities, but when dealing with complex problems, they are prone to errors due to lack of rigor. Therefore, LLMs have limitations in this regard.

Whether LLMs have creativity is still an open question, which also depends on the definition of creativity. Innovation can be divided into incremental innovation and disruptive innovation. Through observation, we can find that LLMs should also possess incremental innovation capability. As for disruptive innovation, such as the establishment of the theory of relativity, whether LLMs can achieve this remains inconclusive. Ilya Sutskever believes that LLMs can perform interpolation, but extrapolation is an open question—this is a similar idea.

LLMs do not have a mechanism corresponding to human consciousness, although we may feel that interacting with LLMs is like interacting with a real person. Consciousness refers to the perception of the internal body and external environment felt in the human mind. Consciousness is a state that the human brain is in when awake; other states contrasting with it are deep sleep, coma, and death. Consciousness is subjective—it is each person's self-centered mental activity; for each of us, our own consciousness is continuous, consistent, and stable. At the same time, consciousness is also objective—it corresponds to high-level processing of the human brain's neural system, supported by low-level

processing of the brain's neural system. Global Workspace Theory holds that consciousness is the global broadcast of information within the brain.

6 Conclusion

This artilce discusses the language and thinking capabilities of LLMs from the perspectives of implementation methods and mechanics. We argue that in a certain sense, LLMs have reached or exceeded human levels of language and thinking.

Large language models essentially learn patterns—i.e., regularities—in language data, and can apply these patterns, especially higher-order patterns, to exhibit intelligent behavior.

The capabilities learned by LLMs can be simply summarized as Next Token Prediction, but in reality they are the culmination of highly effective implementation principles and methods. Data scale and model scale also play important roles. Pre-training, through data compression, enables the model to discover regularities in large amounts of diverse data; post-training, through policy modeling, fine-tunes the model to more reasonably complete tasks and conduct reasoning. Gradient descent combined with backpropagation allows model parameters to be adjusted to better represent data patterns. The Transformer architecture, through attention, FFN, and residual mechanisms, can effectively represent hierarchical, diverse, and flexibly composable patterns in data.

Complex mechanics exist within LLMs. Analysis tools such as SAE and CLT have discovered that models truly contain large numbers of features at different levels, and these features form circuits that can be memorized during learning and retrieved during prediction.

Large language models actually represent a general technology—a paradigm—for learning patterns in sequential data. They can be applied not only to language data, but also to other types of data such as images.

References

[1] Mikhail Belkin, Daniel Hsu, Siyuan Ma, and Soumik Mandal. Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32):15849–15854, 2019.

[2] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition, 2022.

[3] Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why language models hallucinate, 2025.

[4] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[5] Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*, 2016.

[6] Trung Quoc Luong, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. Reft: Reasoning with reinforced fine-tuning, 2024.

[7] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization, 2017.

[8] Shaohua Zhang, Yuan Lin, and Hang Li. Memory retrieval and consolidation in large language models through function tokens, 2025.

[9] Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, and Junyang Lin. Stabilizing reinforcement learning with llms: Formulation and practices, 2025.